Name-Bhakare Purva Shivaji

Roll no-08

Practiacl no-7

```
# Logistic Regression with Metrics and Graphs
import zipfile
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score
from sklearn.metrics import recall_score, f1_score, confusion_matrix
# 1. ZIP FILE PATH
zip_path = r"C:\Users\PVGIT\Downloads\archive (14).zip"
extract_path = r"C:\Users\PVGIT\Downloads\dataset"
# Extract ZIP file
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)
# 2. FIND CSV FILE
csv_file = None
for root, dirs, files in os.walk(extract_path):
    for file in files:
        if file.endswith(".csv"):
            csv_file = os.path.join(root, file)
            break
    if csv_file:
        break
print("CSV file:", csv_file)
```

```
# 3. LOAD DATA
data = pd.read_csv(csv_file)
# Remove missing values
data = data.dropna()
print("\nDataset:")
print(data.head())
# 4. FEATURES AND TARGET
# Last column is target
target = data.columns[-1]
print("\nTarget column:", target)
X = data.drop(target, axis=1)
y = data[target]
# Convert categorical columns to numbers
X = pd.get_dummies(X, drop_first=True)
# 5. TRAIN TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
# 6. FEATURE SCALING
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# 7. LOGISTIC REGRESSION
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
# 8. PREDICTION
y_pred = model.predict(X_test)
```

```
# 9. CALCULATE METRICS
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)
recall = recall_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)
f1 = f1_score(
    y_test,
    y_pred,
    average="weighted",
    zero_division=0
)
# 10. PRINT RESULTS
print("\n================================")
print("     LOGISTIC REGRESSION")
print("================================")
print("Accuracy  :", accuracy)
print("Precision :", precision)
print("Recall    :", recall)
print("F1-Score  :", f1)
# 11. CONFUSION MATRIX
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
```

```
print(cm)
# 12. CONFUSION MATRIX GRAPH
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)
plt.title("Logistic Regression - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
# 13. METRICS GRAPH
metrics = ["Accuracy", "Precision", "Recall", "F1-Score"]
values = [accuracy, precision, recall, f1]
plt.figure(figsize=(8, 5))
bars = plt.bar(
    metrics,
    values,
    color=["blue", "green", "orange", "red"]
)
plt.title("Logistic Regression Performance")
plt.xlabel("Metrics")
plt.ylabel("Score")
plt.ylim(0, 1)
# Show values on top of bars
for bar, value in zip(bars, values):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.02,
```

```
        f"{value:.2f}",
        ha="center"
    )
plt.show()
```

OUTPUT-

Dataset:

```
         Country Name        PPT          GDP   PRPE     OOCP       ESE  \
0             Albania    2893654  1.321986e+10   0.73     7097    333291
1          Arab World  384222592  2.889750e+12   6.42  6827541  30972246
2  United Arab Emirates    9086139  4.019580e+11   0.22    14611    411040
3          Azerbaijan    9535079  7.519801e+10   0.16    22821    949294
4             Burundi   10816860  3.093647e+09  24.25    69246    583308

       EPE  UNEMP    LEB  TDP
0   195720  16.10  77.83    5
1  45126932  11.52  70.57    6
2    409776   3.60  77.37    5
3    517708   5.20  70.76    4
4   2046794   6.90  56.69    6
```

Target column: TDP

```
================================
    LOGISTIC REGRESSION
================================
Accuracy  : 0.6818181818181818
Precision : 0.4648760330578512
Recall    : 0.6818181818181818
F1-Score  : 0.5528255528255529
```

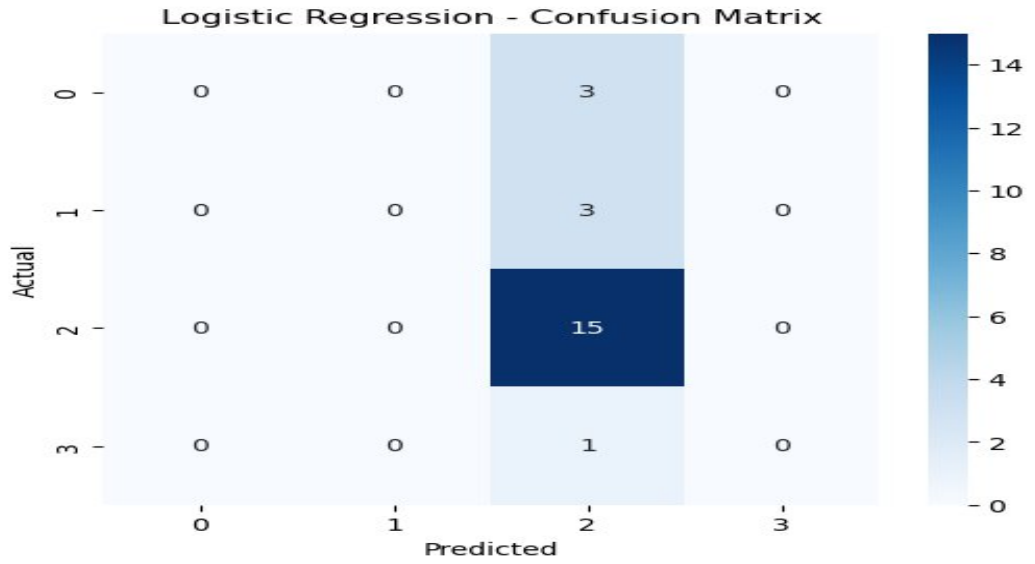
Confusion Matrix:

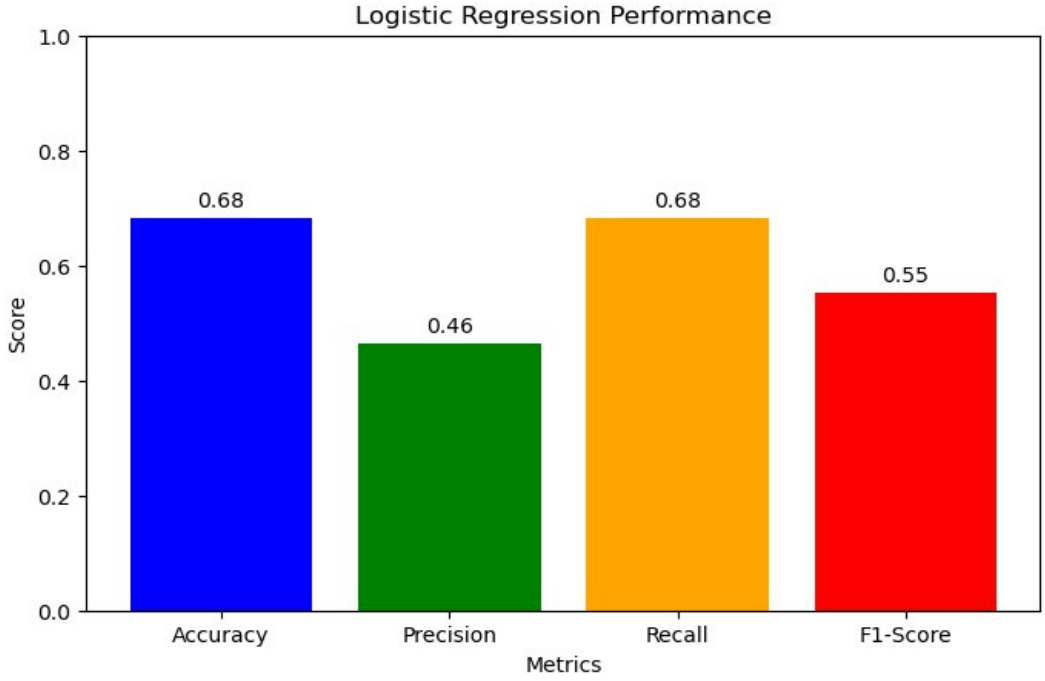
[[0 0 3 0]

[0 0 3 0]

[0 0 15 0]

[0 0 1 0]]





==========

KNN CLASSIFICATION

==========

```
import zipfile
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
# 1. ZIP FILE PATH
zip_path = r"C:\Users\PVGIT\Downloads\archive (15).zip"
extract_path = r"C:\Users\PVGIT\Downloads\knn_data"
# 2. EXTRACT ZIP FILE
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)
print("Dataset extracted successfully!")
# 3. FIND CSV FILE
csv_file = None
for root, folders, files in os.walk(extract_path):
    for file in files:
        if file.endswith(".csv"):
            csv_file = os.path.join(root, file)
            break
    if csv_file:
        break
```

```
print("CSV file:", csv_file)
# 4. LOAD DATASET
data = pd.read_csv(csv_file)
print("\nOriginal dataset shape:")
print(data.shape)
print("\nColumns:")
print(data.columns.tolist())
# 5. REMOVE EMPTY COLUMNS
data = data.dropna(axis=1, how="all")
# 6. USE TRS AS TARGET
target = "TRS"
print("\nTarget column:", target)
# Remove rows where TRS is missing
data = data.dropna(subset=[target])
# 7. CONVERT TRS INTO CLASSES
# Median value of TRS
median_trs = data[target].median()
print("\nTRS Median:", median_trs)
# 0 = Low TRS
# 1 = High TRS
data["TRS_Class"] = (data[target] >= median_trs).astype(int)
print("\nClass distribution:")
print(data["TRS_Class"].value_counts())
# 8. SELECT FEATURES
features = [
    "Benzene",
    "CO",
    "PM10",
    "PM2.5",
```

```
    "NO2",
    "O3",
    "SO2",
    "Toluene"
]
X = data[features]
y = data["TRS_Class"]
# 9. SAMPLE DATA
# KNN is slow with millions of rows
# Use 50,000 samples
if len(X) > 50000:
    sample = pd.concat([X, y], axis=1)
    sample = sample.sample(
        n=50000,
        random_state=42
    )
    X = sample[features]
    y = sample["TRS_Class"]
print("\nData used for KNN:")
print(X.shape)
# 10. HANDLE MISSING VALUES
imputer = SimpleImputer(strategy="median")
X = imputer.fit_transform(X)
# 11. TRAIN TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
# 12. FEATURE SCALING
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# 13. KNN MODEL
knn = KNeighborsClassifier(
    n_neighbors=5
)
knn.fit(X_train, y_train)
# 14. PREDICTION
y_pred = knn.predict(X_test)
# 15. CALCULATE METRICS
accuracy = accuracy_score(
    y_test,
    y_pred
)
precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)
recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)
f1 = f1_score(
    y_test,
    y_pred,
```

```
    zero_division=0
)
# 16. PRINT RESULT
print("\n====================================")
print("          KNN RESULTS")
print("====================================")
print("Accuracy  :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall    :", round(recall, 4))
print("F1 Score  :", round(f1, 4))
# 17. CONFUSION MATRIX
cm = confusion_matrix(
    y_test,
    y_pred
)
print("\nConfusion Matrix:")
print(cm)
# 18. CONFUSION MATRIX GRAPH
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Low TRS", "High TRS"],
    yticklabels=["Low TRS", "High TRS"]
)
plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
# 19. PERFORMANCE GRAPH
metrics = [
    "Accuracy",
    "Precision",
    "Recall",
    "F1 Score"
]
values = [
    accuracy,
    precision,
    recall,
    f1
]
plt.figure(figsize=(8, 5))
bars = plt.bar(
    metrics,
    values,
    color=[
        "blue",
        "green",
        "orange",
        "red"
    ]
)
plt.title("KNN Performance")
plt.xlabel("Metrics")
plt.ylabel("Score")
plt.ylim(0, 1)
```

```
# Display values
for bar, value in zip(bars, values):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        value + 0.02,
        f"{value:.2f}",
        ha="center"
    )
plt.show()
```

OUTPUT-

```
Dataset extracted successfully!
CSV file: C:\Users\PVGIT\Downloads\knn_data\Dados_SP_AR_Qualidade.csv
Original dataset shape:
(3445260, 11)
Columns:
['Datetime', 'Station', 'Benzene', 'CO', 'PM10', 'PM2.5', 'NO2', 'O3', 'SO2', 'Toluene', 'TRS']
Target column: TRS
TRS Median: 2.0
Class distribution:
TRS_Class
1    33138
0    28492
Name: count, dtype: int6
Data used for KNN:
(50000, 8)
====================================
        KNN RESULTS
====================================
Accuracy  : 0.6866
Precision : 0.704
```

Recall : 0.7176

F1 Score : 0.7107

Confusion Matrix:

[[3016 1619]

[1515 3850]]

